Adam Lee
Dr. Levent Yilmaz
COMP3270
3/23/2025

Homework 3

1.
Initial heap:
A = [6, 8, 9, 10, 12, 16, 15, 13, 14, 19, 18, 17]
Insert 6 at the end:
A = [6, 8, 9, 10, 12, 16, 15, 13, 14, 19, 18, 17, 6]
Compare with parent (16), swap
Compare with parent (9), swap
Compare with parent (6), stop
Answer: A = [6, 8, 6, 10, 12, 9, 15, 13, 14, 19, 18, 17, 16]

2. Sort A (O(n log n))
Traverse once and Keep first element
For each next element, only keep it if it's different from previous
Result: duplicates removed in O(n log n)

3. Sort S (O(n log n))
Traverse and count consecutive duplicates:
Track current count and max count
Update mode when a higher count is found
Final: return element with highest frequency

4. Any comparison-based sorting algorithm can be made stable by modifying the comparison operation. Instead of comparing elements only by their values, we compare pairs of the form (value, original index). If two elements have equal values, the one with the smaller original index is considered smaller. This preserves the relative order of equal elements, making the algorithm stable. Since each comparison still takes constant time, the overall asymptotic running time of the algorithm remains unchanged.

5. a) Recursion Tree
Initial:
A = [10, 3, 9, 4, 8, 5, 7, 6], p=1, r=8, k=2

Partition (pivot = 6):
A = [3, 4, 5, 6, 8, 9, 7, 10]
q = 4, pivotDistance = 4
k = 2 < 4, go left
Next call:
A = [3, 4, 5], p=1, r=3, k=2
Partition (pivot = 5):
A = [3, 4, 5]
q = 3 to pivotDistance = 3
k = 2 < 3 = go left
Next call:
A = [3, 4], p=1, r=2, k=2
Partition (pivot = 4):
A = [3, 4]
q = 2 to pivotDistance = 2
k = 2 to return 4

B) Base case/Recurrences

1st base case, Array has only one element.
- Check p == r
Return A[p]
Recurrence: T(1st) = c
2nd base case, Pivot is the k-th smallest element
- partition, Compute pivotDistance
If k == pivotDistance, return pivot
Recurrence: T(2nd)= 20n + c
Non-base case:
-partition (20n)
- recurse on one side
Worst Case: T(n) = T(n-1) + 20n + c
Best Case: T(n) = T(n/2) +20n + c

6.

A = [19, 6, 10, 7, 16, 17, 13, 14, 12, 9]
After counting, C = [0,0,0,0,0,0,1,1,0,1,1,0,1,1,1,0,1,1,0,1]
Prefix sum(C), C = [0,0,0,0,0,0,1,2,2,3,4,4,5,6,7,7,8,9,9,10]
Output B: B = [6, 7, 9, 10, 12, 13, 14, 16, 17, 19]

7.

Initial:
[4567, 3210, 2345, 4321, 5678]
Pass 1 (1's digit):
[3210, 4321, 2345, 4567, 5678]
Pass 2 (10's digit):
[3210, 4321, 2345, 4567, 5678]
Pass 3 (100's digit):
[3210, 4321, 2345, 4567, 5678]
Pass 4 (1000's digit):
[2345, 3210, 4321, 4567, 5678]

8.

For n = 15:
Bucket i stores: [i/15, (i+1)/15)
Bucket0: [0, 1/15)
Bucket1: [1/15, 2/15)
Bucket2: [2/15, 3/15)
Bucket3: [3/15, 4/15)
Bucket4: [4/15, 5/15)
Bucket5: [5/15, 6/15)
Bucket6: [6/15, 7/15)
Bucket7: [7/15, 8/15)
Bucket8: [8/15, 9/15)
Bucket9: [9/15, 10/15)
Bucket10: [10/15, 11/15)
Bucket11: [11/15, 12/15)
Bucket12: [12/15, 13/15)
Bucket13: [13/15, 14/15)
Bucket14: [14/15, 1)

General case (n buckets)
Bucket0: [0, 1/n)
Bucket1: [1/n, 2/n)
.....
Bucket(n−2): [(n−2)/n, (n−1)/n)
Bucket(n−1): [(n−1)/n, 1)

9.

Part A, Unions: Final parent array P (index = element, value = parent):

P = [1, 1, 14, 3, 3, 3, 1, 1, 8, 3, 3, 3, 3, 1, 14, 18, 16, 19, 20, 1]

Part B, by Height: attach short tree under taller, If equal = second under first

P = [1, 1, 3, 3, 3, 3, 1, 1, 8, 3, 3, 3, 3, 3, 14, 16, 16, 16, 16, 16]

Part C, by Size: attach smaller tree under larger, if equal = second under first

P = [1, 1, 3, 3, 3, 3, 1, 1, 8, 3, 3, 3, 3, 3, 14, 16, 16, 16, 16, 16]

Part D, Path Compression:

Deepest chain is:
17 to 16 to 18 to 19 to 20 to 1

After Find(17) with path compression:

All nodes on that path point directly to root (1)

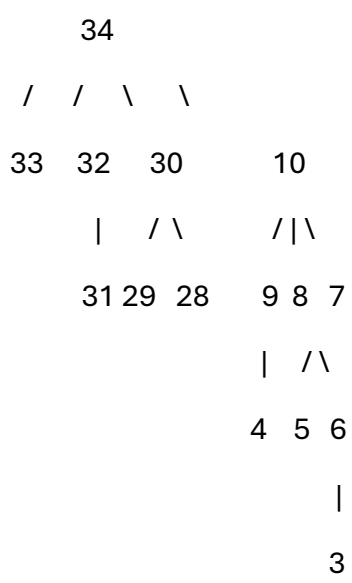
Updated P: P = [1, 1, 14, 3, 3, 3, 1, 1, 8, 3, 3, 3, 3, 1, 14, 1, 1, 1, 1, 1]

10.

B_2 tree(root =38)



B_4 tree(root = 34)



<u>After EXTRACT_MAX (remove 38)</u>

Remove the root 38, and take its children

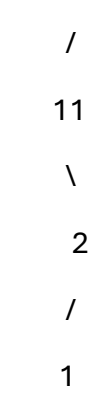
Children (in reverse order for BQ)

- B_1 rooted at 22
- B_0 rooted at 37

<u>Remaining trees:</u>

B_4(unchanged)

B_1(root = 22)



B_0(root = 37), the tree is just 37.